# PyRepair Agent

## 测试诊断与候选补丁生成智能体

MVP 产品与工程设计说明

目标：14天完成可安装、可评测、可接入CI的第一版

**状态：设计已批准**

DESIGN SPECIFICATION / VERSION 1.0

2026-08-12

# 目录

本设计聚焦两周MVP：产品边界、CLI、模块职责、安全约束、测试评测、GitHub Actions及学习路线。


1. 项目目标 4

2. 第一版范围 4

2.1 必须支持 4

2.2 暂不支持 4

3. CLI设计 5

3.1 默认用法 5

3.2 可选高级参数 5

4. 总体架构 5

5. 模块职责 6

5.1 CLI 6

5.2 RepairCoordinator 6

5.3 WorkspaceManager 6

5.4 TestRunner 7

5.5 PytestParser 7

5.6 Tool与ToolRegistry 7

5.7 DeepSeekProvider 7

5.8 RepairAgent 7

5.9 PatchPolicyGate 7

5.10 ReviewAgent 8

5.11 Verifier 8

5.12 RunRecorder 8

6. 数据接口 8

7. 错误处理 9

7.1 正常业务失败 9

7.2 基础设施异常 9

8. 测试与评测策略 9

8.1 单元测试 . . . 9
8.2 FakeLLM集成测试 . . . 10
8.3 Live Smoke Test . . . 10
8.4 Benchmark . . . 10

**9. GitHub Actions设计** **10**

**10. 建议目录** **11**

**11. 两周里程碑** **11**

第1周：打通闭环 . . . 11
第2周：质量与展示 . . . 11

**12. Hello Agents学习路线** **12**

12.1 开发前必须完成 . . . 12
12.2 开发中按需选读 . . . 12
12.3 项目第一版后再学 . . . 12

**13. 完成标准** **12**

# 1. 项目目标

PyRepair Agent是面向Python项目的测试诊断与候选补丁生成Agent。开发者在目标项目根目录执行：

```
pyrepair fix
```

系统执行以下闭环：

1. 在安全临时工作区运行pytest并记录初始失败。
2. 将pytest输出解析为结构化故障信息。
3. 使用DeepSeek和受限工具定位根因、生成最小修改。
4. 通过确定性安全规则检查Patch。
5. 使用独立的Review角色审查Diff。
6. 重跑目标测试和全量测试。
7. 验证通过后输出候选Patch、结构化报告与完整轨迹。
8. 达到最大尝试次数仍失败时丢弃临时修改并输出失败原因。

PyRepair不试图替代Claude Code等通用编程Agent。其差异化定位是：把测试修复收敛为可接入CI、安全约束明确、结果可复现评测的专用工作流。

# 2. 第一版范围

## 2.1 必须支持

- 可安装的Python CLI，默认入口为`pyrepair fix`。
- 默认处理当前目录下的Python/Git项目。
- 固定使用`python -m pytest`，不接受任意Shell测试命令。
- 默认模型为`deepseek-v4-flash`，通过配置可切换`deepseek-v4-pro`。
- 支持DeepSeek Tool Calling、Thinking模式、超时、重试和Token统计。
- 受限工具：读取文件、搜索代码、精确替换、查看Diff、运行目标测试。
- 临时安全工作区、失败回滚和候选Patch输出。
- 确定性Patch安全检查、独立Review和确定性Verifier。
- 目标测试验证、全量测试验证和测试回归检测。
- 最多3轮修复，超限后明确失败。
- 输出`fix.patch`、`report.json`、`trace.jsonl`和`pytest.log`。
- 不少于15个真实调用DeepSeek的Bug评测任务。
- 普通CI和一个手动触发的GitHub Actions演示工作流。

## 2.2 暂不支持

- Java、JavaScript等其他语言。

- 通用Issue开发或开放式需求实现。
- 自动修改测试。
- 自动安装目标项目依赖。
- 任意Shell命令执行。
- 自动Commit、Push、创建PR或合并代码。
- MCP、RAG、长期记忆、Agent Teams和Git Worktree。
- Web前端、复杂终端UI和多模型负载均衡。
- 多个失败测试的复杂联合诊断。
- 完整上下文压缩与并行工具执行。

# 3. CLI设计

## 3.1 默认用法

```
pyrepair fix
```

默认行为：

- 目标仓库为当前目录。
- 测试命令为`python -m pytest -q --tb=short`。
- 模型为`deepseek-v4-flash`。
- 最大修复次数为3。
- 在临时副本内修改，不直接改动原项目。
- 成功时只输出候选Patch，不自动应用。

## 3.2 可选高级参数

第一版只保留少量可选参数：

```
pyrepair fix tests/test_account.py
pyrepair fix --repo D:\projects\demo
pyrepair fix --model deepseek-v4-pro
pyrepair fix --max-attempts 2
pyrepair fix --dry-run
pyrepair fix --verbose
```

`--dry-run`只进行诊断和上下文收集，不写入文件。pytest附加参数必须通过白名单构造，不能作为任意命令交给Shell。

# 4. 总体架构

PyRepair分为确定性编排与安全层、LLM推理层、共享基础设施三部分。




设计原则：

- LLM负责分析、选择受限工具和提出最小修改。
- 代码负责流程顺序、权限、安全限制、最大轮数和最终成功判定。
- Review Agent可以提出语义风险，但不能修改代码。
- Verifier不依赖LLM自我声明，测试与规则结果才是完成依据。

# 5. 模块职责

## 5.1 CLI

- 解析`fix`子命令和可选参数。
- 加载配置并创建`RepairRequest`。
- 显示当前阶段、尝试次数、最终状态和产物位置。
- 将业务失败转换为清楚的用户提示，将程序异常映射为非零退出码。

## 5.2 RepairCoordinator

- 固定执行基线测试、诊断、修复、安全检查、Review和验证流程。
- 维护最多3轮的状态。
- 将上一轮策略拒绝、Review发现或测试失败作为下一轮证据。
- 决定成功、继续或失败退出；不把流程控制权完全交给模型。

## 5.3 WorkspaceManager

- 验证目标路径是Python/Git项目。
- 检查并记录原项目基线状态。
- 创建临时安全副本，所有Agent修改只发生在副本内。
- 生成标准Git Diff和候选Patch。
- 失败时丢弃副本；成功时仍不直接修改原项目。

第一版优先使用临时目录复制必要项目文件，避免依赖用户掌握Git分支或Worktree。复制时排除`.git`、虚拟环境、缓存和PyRepair产物目录。

## 5.4 TestRunner

- 通过参数列表而非`shell=True`运行`python -m pytest`。
- 设置工作目录、执行超时并捕获退出码、stdout、stderr和耗时。
- 区分测试失败、测试通过、收集失败、超时和pytest无法启动。

## 5.5 PytestParser

- 将`TestRun`转换为`FailureReport`。
- 第一版聚焦单个失败测试，提取node id、异常类型、Traceback帧、相关文件与摘要。
- 对无法识别的输出保留原始日志并返回“不可结构化解析”，不能猜测不存在的信息。

## 5.6 Tool与ToolRegistry

所有工具实现统一接口，并能转换为DeepSeek可识别的JSON Schema。第一版工具包括：

- `read_file`：只读取工作区内允许的文本文件，并限制返回大小。
- `search_code`：在允许目录中按文本搜索代码。
- `replace_text`：精确匹配旧文本后替换，匹配次数不符合预期时拒绝执行。
- `show_diff`：返回当前修改的标准Diff摘要。
- `run_target_test`：只运行已识别的pytest node id。

所有路径先解析为绝对路径，再确认其位于临时工作区内。工具错误以结构化结果返回模型；基础设施无法工作时才抛出异常。

## 5.7 DeepSeekProvider

- 封装模型名称、API地址、Thinking模式、Tool Calling、超时、重试和Token统计。
- 默认`deepseek-v4-flash`，配置切换`deepseek-v4-pro`时不改Agent Loop。
- 第一版只支持DeepSeek，不提前实现多个供应商适配器。
- API Key只从环境变量读取，不写入配置文件、日志或轨迹。

## 5.8 RepairAgent

- 接收`FailureReport`、相关源码、工具说明和上一轮失败证据。
- 通过Tool Calling逐步读取、搜索、修改和验证。
- 系统提示要求寻找根因、生成最小修改、禁止修改测试或绕过测试。
- 达到单轮最大工具次数时停止并返回失败，不无限循环。

## 5.9 PatchPolicyGate

使用确定性代码检查Diff，至少包括：

- 禁止修改`tests/`、`.env`、密钥、CI配置和依赖锁文件。
- 禁止新增`skip`、`xfail`或删除、弱化断言。

- 禁止路径越界和新增二进制文件。
- 最多修改3个源码文件。
- 新增与删除合计最多80行；超过阈值时拒绝Patch并要求下一轮缩小修改。

策略拒绝是正常业务结果，必须附带违规规则和证据，供下一轮修复参考。

### 5.10 ReviewAgent

- 使用独立消息上下文，只读取故障报告与Diff。
- 判断修改是否针对根因、是否存在明显副作用、是否属于最小必要修改。
- 输出结构化`ReviewResult`，包括是否批准、风险等级、发现和理由。
- Review结果是质量门之一，但不能替代确定性安全规则或测试。

### 5.11 Verifier

依次执行：

1. 原失败目标测试必须通过。
2. 全量测试必须通过。
3. 最终Diff必须再次通过Patch策略。

修改前保存基线失败集合，修改后若出现新的失败测试，即判定为测试回归。只有三项全部通过才能交付成功Patch。

### 5.12 RunRecorder

在`.pyrepair/runs/<run-id>/`中输出：

```
fix.patch
report.json
trace.jsonl
pytest.log
```

- `fix.patch`：候选修改。
- `report.json`：状态、尝试次数、故障、Review、验证和成本摘要。
- `trace.jsonl`：逐事件记录模型调用、工具调用和状态变化；敏感值必须脱敏。
- `pytest.log`：基线、目标和全量测试输出。

## 6. 数据接口

第一版内部模块使用`dataclass`传递明确对象：

- `RepairRequest`：仓库路径、测试目标、模型、最大尝试数、dry-run。
- `TestRun`：命令、退出码、stdout、stderr、耗时、是否超时。
- `FailureReport`：失败node id、异常类型、Traceback和摘要。
- `AgentTurn`：消息、工具调用、工具结果、Token和停止原因。

- `PatchCheck`：是否允许、违规项、修改文件和行数统计。
- `ReviewResult`：是否批准、风险、发现和理由。
- `VerificationResult`：目标测试、全量测试、回归和策略结果。
- `RunResult`：最终状态、尝试次数、产物路径和失败原因。

内部传递对象不等于传递JSON。输出报告时可以用`dataclasses.asdict()`序列化为JSON。未来只在外部边界需要强验证时局部引入Pydantic，例如用户配置、Benchmark用例文件、Web API输入或LLM结构化输出。

# 7. 错误处理

## 7.1 正常业务失败

以下情况作为结构化结果返回，由Coordinator决定重试或结束：

- 测试仍失败。
- Patch被策略拒绝。
- Review不批准。
- 全量测试出现回归。
- 达到最大尝试次数。

## 7.2 基础设施异常

以下情况抛出明确异常并停止：

- 配置或API Key缺失。
- 目标路径无效。
- pytest无法启动或超时。
- DeepSeek API在规定重试后仍不可用。
- 临时工作区创建、读取或写入失败。

所有异常信息必须说明发生阶段、可行动的原因和日志位置，不能只显示Python堆栈。

# 8. 测试与评测策略

## 8.1 单元测试

完全离线、每次开发运行，覆盖：

- pytest输出解析。
- 路径与Diff安全规则。
- 配置与Tool Registry。
- 数据对象和报告序列化。
- 超时、退出码和异常分支。

### 8.2 FakeLLM集成测试

使用预设Tool Call的FakeLLM串通完整流程，不调用DeepSeek、不产生费用，覆盖：

- Agent Loop与工具结果回填。
- 成功修复路径。
- 策略拒绝、Review拒绝和回滚。
- 目标测试与全量测试验证。
- 最大轮数结束。

### 8.3 Live Smoke Test

通过pytest marker手动运行一个简单真实模型任务：

```
pytest -m live
```

用于确认DeepSeek鉴权、消息格式、Tool Calling和Thinking模式兼容。普通`pytest`默认跳过该测试。

### 8.4 Benchmark

真实调用DeepSeek运行不少于15个Bug任务，任务答案不能泄漏到提示词中。任务至少覆盖：

- 条件判断错误。
- 边界值错误。
- 异常处理错误。
- 状态更新错误。
- 数据转换错误。

统计：

- 最终修复成功率。
- 首轮修复成功率。
- 平均尝试次数。
- 平均耗时和Token。
- 全量测试回归数。
- 安全规则违规与拦截数。

不预先承诺高成功率。可信的任务、零虚假成功和可复现数据优先于漂亮数字。

## 9. GitHub Actions设计

第一版包含两个工作流：

- 普通CI：安装PyRepair并运行离线测试，不读取DeepSeek API Key。
- PyRepair演示：手动触发或在受信任仓库环境中运行，调用同一个`pyrepair fix`并上传Patch与报告作为Artifact。

来自外部Fork的Pull Request不能直接获得API Secret。第一版不自动提交、评论或合并，避免密钥泄漏和恶意代码执行风险。

# 10. 建议目录

```
pyrepair-agent/
├─ pyproject.toml
├─ README.md
├─ src/pyrepair/
│  ├─ cli.py
│  ├─ config.py
│  ├─ models.py
│  ├─ coordinator.py
│  ├─ workspace.py
│  ├─ testing.py
│  ├─ pytest_parser.py
│  ├─ tools.py
│  ├─ deepseek.py
│  ├─ repair_agent.py
│  ├─ policy.py
│  ├─ review_agent.py
│  ├─ verifier.py
│  └─ recorder.py
├─ tests/
├─ benchmarks/
│  ├─ run_benchmark.py
│  └─ cases/
├─ examples/buggy_bank/
├─ docs/
└─ .github/workflows/
```

第一版使用单层模块，避免为了形式创建过多嵌套包。模块增长后再按领域分包。

# 11. 两周里程碑

## 第1周：打通闭环

- Day 1：学习Hello Agents 7.5.1-7.5.2；Git基础；仓库、包与CLI入口。
- Day 2：`TestRun`、pytest执行、超时和单元测试。
- Day 3：`FailureReport`与常见pytest输出解析。
- Day 4：DeepSeek Tool Calling Loop与FakeLLM。
- Day 5：受限文件、搜索、替换、Diff和目标测试工具。
- Day 6：临时工作区、Patch策略和失败回滚。
- Day 7：在一个自制Bug项目上完成真实模型修复闭环。

## 第2周：质量与展示

- Day 8：Review Agent。
- Day 9：Verifier、测试回归检测和三轮重试。
- Day 10：Patch、JSON报告、JSONL轨迹和CLI错误信息。

- Day 11：制作至少15个Benchmark任务。
- Day 12：运行V4 Flash评测并记录数据。
- Day 13：GitHub Actions普通CI与手动演示。
- Day 14：README、架构图、评测报告、演示视频和简历材料。

如果进度落后，优先删除Flash/Pro对比、复杂CLI参数、多失败测试、自动PR、并行工具和上下文压缩；不能删除安全规则、确定性验证、评测和文档。

# 12. Hello Agents学习路线

PyRepair MVP不要求先学完第8章以后全部内容。

## 12.1 开发前必须完成

- 7.5.1：工具基类、统一接口、工具注册与JSON Schema。
- 7.5.2：自定义工具开发与统一执行。
- 7.5.4中的ToolChain思想：理解确定性流程如何串联多个工具。

7.5.3多源搜索只理解统一后端和容错思想；异步工具只理解原理，不在MVP实现。

## 12.2 开发中按需选读

- 第9章上下文工程：重点阅读上下文构建、安全文件访问和代码库维护助手，帮助优化给Repair Agent的代码上下文。
- 第12章智能体性能评估：重点理解任务集、指标、错误分析和评估报告，用于Benchmark设计。
- 第13章13.2 Pydantic：项目后期需要验证外部配置或结构化LLM输出时再读。
- 第14章TODO驱动研究与职责划分：只在实现Coordinator和Review角色时参考流程设计，不必完整学习前端和研究工具。

## 12.3 项目第一版后再学

- 第8章记忆与RAG：PyRepair单次修复不需要长期记忆或向量检索。
- 第10章MCP/A2A/ANP：第一版工具直接注册即可，不需要远程协议。
- 第11章Agentic-RL、SFT和GRPO：本项目调用现成模型，不训练模型。
- 第13章完整多Agent旅行助手和前端。
- 第14章完整深度研究系统。

跳过这些章节不会破坏MVP。项目完成后再补第8-14章，能把已经遇到的工程问题与课程概念对应起来，学习效果更好。

# 13. 完成标准

两周版本完成必须同时满足：

- `pyrepair fix`可以从干净环境安装并运行。
- 至少一个真实DeepSeek任务完成安全修复闭环。
- 策略能够拦截修改测试、路径越界和超量修改。
- 成功Patch同时通过目标测试、全量测试和最终策略检查。
- 失败尝试不会修改原项目，并能输出原因。
- 普通测试完全离线，Live测试明确标记。
- 不少于15个Benchmark任务及真实结果。
- 公开GitHub仓库包含清晰提交记录、README、架构说明和演示材料。

## 设计结论

PyRepair以确定性安全与验证代码包围LLM推理：模型提出修改，规则控制边界，测试决定是否完成。第一版成功标准不是夸大的通用能力，而是一个可安装、可复现、可测量的Python测试修复闭环。